

1. Executive Summary

This project focused on implementing browser-side security controls through the configuration of HTTP security headers on an Nginx web server. The objective was to deploy a containerized Nginx instance, configure defensive security headers, and verify their successful implementation using command-line validation techniques.

The implemented security controls protect web applications against common client-side attacks such as Clickjacking, MIME-type sniffing, and Cross-Site Scripting (XSS). The project was completed within a local laboratory environment using Docker and Ubuntu Linux.

2. Project Objectives

The primary objectives of this project were:

- ❖ Deploy a standard Nginx web server using Docker.
- ❖ Configure browser security headers at the proxy boundary.
- ❖ Implement protection against Clickjacking attacks.
- ❖ Prevent MIME-sniffing vulnerabilities.
- ❖ Restrict unauthorized script execution through Content Security Policy (CSP).
- ❖ Validate the applied security controls using HTTP header inspection.

3. Lab Environment

Component	Technology Used
-----------	-----------------

- | | |
|----------------------|----------------------|
| ❖ Host Platform | ❖ VMware Workstation |
| ❖ Operating System | ❖ Ubuntu Linux |
| ❖ Container Platform | ❖ Docker |

- ❖ Web Server
- ❖ Validation Tool
- ❖ Browser
- ❖ Nginx
- ❖ curl
- ❖ Firefox

4. Deployment Procedure

The following deployment steps were performed:

1. Installed Docker on Ubuntu Linux.
2. Pulled the official Nginx Docker image.
3. Created the project directory.
4. Created the Nginx configuration file.
5. Created a test HTML page.
6. Started the Nginx container.
7. Verified accessibility through a browser.

Screenshot 1: Nginx Docker image

```
gracious@gracious-Virtual-Platform:~$ sudo docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
c82e83b01f69: Pull complete
72c03230f136: Pull complete
8da80f8205ea: Pull complete
3461bb328618: Pull complete
ac5e3151b8c0: Pull complete
99181f19640f: Pull complete
868d78dceaed: Pull complete
4b84eff91c61: Download complete
bcac76e1b5c9: Download complete
Digest: sha256:6415da96b72a2f6ff433053df1da4bd507e45bc44b2c83dbac0ddeea80c6f066
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Figure 1: the downloading and the pulling of the official Nginx Docker image

Screenshot 2: Project Directory Structure

```
gracious@gracious-VMware-Virtual-Platform:~/project-guard-post$ ls -la ~/project-guard-post
total 16
drwxrwxr-x 3 gracious gracious 4096 Jun 18 11:57 .
drwxr-x-- 16 gracious gracious 4096 Jun 18 11:51 ..
drwxrwxr-x 2 gracious gracious 4096 Jun 18 12:12 html
-rw-rw-r-- 1 gracious gracious 396 Jun 18 11:55 nginx.conf
```

Figure 2: Project directory containing nginx.conf and HTML web content.

Screenshot 3: Running Docker Container

```
gracious@gracious-VMware-Virtual-Platform:~/project-guard-post$ sudo docker ps
[sudo] password for gracious:
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
33f6f5ff650a   nginx         "/docker-entrypoint...." About an hour ago Up About an hour 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
guard-post
```

Figure 3: Active Docker container running the Nginx web server

5. Security Header Configuration

The following Nginx configuration command was implemented to enforce browser security controls.

```
gracious@gracious-VMware-Virtual-Platform:~/project-guard-post$ cat nginx.conf
events {}

http {

    server {

        listen 80;

        server_name localhost;

        location / {

            root /usr/share/nginx/html;
            index index.html;

            add_header X-Frame-Options "DENY" always;

            add_header X-Content-Type-Options "nosniff" always;

            add_header Content-Security-Policy "default-src 'self';" always;

        }

    }

}
```

Figure 4: Hardened Nginx configuration implementing security response headers.

6. Security Controls Implemented

X-Frame-Options

- ❖ `add_header X-Frame-Options "DENY" always;`

Purpose:

Prevents the website from being embedded inside iframes, thereby mitigating Clickjacking attacks.

X-Content-Type-Options

- ❖ `add_header X-Content-Type-Options "nosniff" always;`

Purpose:

Prevents browsers from MIME-type sniffing and executing content as unintended file types.

Content-Security-Policy

- ❖ `add_header Content-Security-Policy "default-src 'self';" always;`

Purpose:

Restricts browser resource loading to trusted sources and helps mitigate Cross-Site Scripting (XSS).

7. Validation and Testing

After deployment, the application was accessed through a web browser.

URL:

<http://localhost:8080>

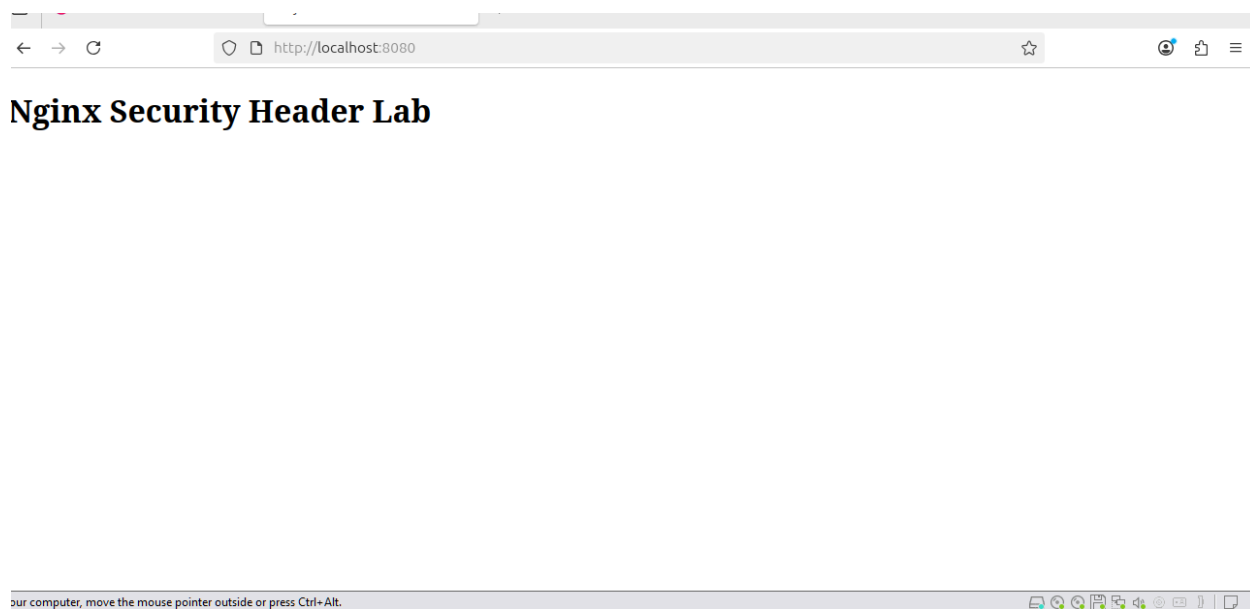


Figure 5 Successful rendering of the hosted web application through the Nginx container.

```
gracious@gracious-VMware-Virtual-Platform:~/project-guard-post$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Server: nginx/1.31.2
Date: Thu, 18 Jun 2026 11:28:55 GMT
Content-Type: text/html
Content-Length: 133
Last-Modified: Thu, 18 Jun 2026 10:58:09 GMT
Connection: keep-alive
ETag: "6a33cf41-85"
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'self';
Accept-Ranges: bytes
```

Figure 6: Verification that all configured security headers are present in the HTTP response.

8. Security Assessment

The following controls were successfully implemented:

Security Control

Threat Mitigated

X-Frame-Options Clickjacking

X-Content-Type-Options MIME Sniffing

Content-Security-Policy Cross-Site Scripting (XSS)

The implementation demonstrates browser-side hardening techniques commonly used in modern web application security.

9. Challenges Encountered

A permission issue was encountered when executing Docker commands without elevated privileges.

- ❖ Error:
- ❖ permission denied while trying to connect to the docker API

Resolution

Docker commands were executed using sudo privileges, allowing successful container management and deployment.

10. Conclusion

Project Guard-Post was successfully completed by deploying a Dockerized Nginx web server and implementing three critical browser security headers.

Validation confirmed that:

- ❖ X-Frame-Options was active.
- ❖ X-Content-Type-Options was active.
- ❖ Content-Security-Policy was active.

The project provided practical experience in Docker containerization, Nginx administration, web application hardening, and HTTP security validation techniques.

